



# Python: Do Zero ao Avançado

Curso completo com 10 módulos

Fundamentos • OOP • APIs • boto3 • Bedrock • Automação AWS

CloudTrilhas — [cloudtrilhas.com.br](https://cloudtrilhas.com.br)

Por Ricardo Simines Scopim

 Python — Módulo 01

# Fundamentos: Variáveis, Tipos e Operadores

Python é uma linguagem de alto nível, interpretada e com tipagem dinâmica. Ideal para automação, dados, web e IA.

## 1. Variáveis e Tipos de Dados

### Variáveis BÁSICO

Em Python, variáveis são criadas na atribuição. Não é necessário declarar tipo.

```
# Tipos primitivos
nome = "Ricardo"      # str
idade = 30            # int
altura = 1.75         # float
ativo = True          # bool
nulo = None           # NoneType

# Verificar tipo
print(type(nome))      # <class 'str'>
print(isinstance(idade, int)) # True
```

### f-strings FORMATAÇÃO

F-strings são a forma moderna e eficiente de formatar strings em Python 3.6+.

```
# f-strings (Python 3.6+)
nome = "Ricardo"
idade = 30
print(f"Olá, {nome}! Você tem {idade} anos.")

# Expressões dentro de f-strings
preco = 49.90
print(f"Preço: R${preco:.2f}")
print(f"{'Python':^20}")      # centralizado
print(f"{1024:_}")           # 1_024 (separador)
```

## 2. Operadores

### Aritméticos

#### OPERADORES

```
a, b = 10, 3
print(a + b)    # 13
print(a - b)    # 7
print(a * b)    # 30
print(a / b)    # 3.333...
print(a // b)   # 3 (divisão inteira)
print(a % b)    # 1 (módulo)
print(a ** b)   # 1000 (potência)
```

### Comparação e Lógicos

#### OPERADORES

```
# Comparação
x = 5
print(x == 5)    # True
print(x != 3)    # True
print(x > 3)     # True
print(x >= 5)    # True

# Lógicos
print(x > 3 and x < 10) # True
print(x > 10 or x < 3)  # False
print(not (x > 10))     # True
```

## 3. Entrada e Conversão de Tipos

## input / casting I/O

```
# Entrada do usuário (sempre retorna str)
nome = input("Seu nome: ")
idade = int(input("Sua idade: "))
peso = float(input("Seu peso: "))

# Conversões (casting)
numero_str = "42"
numero_int = int(numero_str)      # 42
numero_float = float(numero_str)  # 42.0
texto = str(123)                  # "123"
booleano = bool(1)                # True (0 = False)
```

 Python — Módulo 02

# Estruturas de Controle: if, for, while

Controle o fluxo de execução do programa com condicionais e loops.

## 1. Condicionais

if / elif / else CONDICIONAL

```
# Estrutura completa
nota = 7.5

if nota >= 9:
    conceito = "A"
elif nota >= 7:
    conceito = "B"
elif nota >= 5:
    conceito = "C"
else:
    conceito = "D"

print(f"Nota {nota} = Conceito {conceito}")

# Operador ternário
status = "Aprovado" if nota >= 7 else "Reprovado"

# Match/case (Python 3.10+)
match conceito:
    case "A":
        print("Excelente!")
    case "B":
        print("Bom trabalho!")
    case _:
        print("Continue estudando!")
```

## 2. Loops

**for** **LOOP**

```
# Iterando sobre sequências
frutas = ["maçã", "banana", "uva"]
for fruta in frutas:
    print(fruta)

# range()
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 11, 2): # 1, 3, 5, 7, 9
    print(i)

# enumerate (índice + valor)
for i, fruta in enumerate(frutas):
    print(f"{i}: {fruta}")
```

**while** **LOOP**

```
# Loop com condição
contador = 0
while contador < 5:
    print(f"Contagem: {contador}")
    contador += 1

# break e continue
for i in range(10):
    if i == 3:
        continue # pula o 3
    if i == 7:
        break    # para no 7
    print(i)

# while True com break
while True:
    resp = input("Sair? (s/n): ")
    if resp.lower() == "s":
        break
```

 Python — Módulo 03

# Funções e Módulos

Funções organizam código em blocos reutilizáveis. Módulos permitem dividir o projeto em arquivos.

## 1. Funções

### def FUNÇÕES

```
# Função básica
def saudacao(nome, saudacao="Olá"):
    """Retorna uma saudação personalizada."""
    return f"{saudacao}, {nome}!"

print(saudacao("Ricardo"))
print(saudacao("Ana", "Bom dia"))

# *args e **kwargs
def soma(*numeros):
    return sum(numeros)

print(soma(1, 2, 3, 4)) # 10

def criar_perfil(**dados):
    return {k: v for k, v in dados.items()}

perfil = criar_perfil(nome="Ricardo", cargo="DevOps")
```

## lambda FUNÇÕES

Funções anônimas de uma linha.

```
# Lambda

dobro = lambda x: x * 2

print(dobro(5)) # 10


# Com sorted

alunos = [("Ana", 8), ("Bia", 9), ("Carlos", 7)]

ordenados = sorted(alunos, key=lambda x: x[1], reverse=True)


# Com map e filter

numeros = [1, 2, 3, 4, 5]

pares = list(filter(lambda x: x % 2 == 0, numeros))

dobros = list(map(lambda x: x * 2, numeros))
```

## List Comprehension PYTHONIC

Forma concisa e eficiente de criar listas.

```
# Básico

quadrados = [x**2 for x in range(10)]


# Com filtro

pares = [x for x in range(20) if x % 2 == 0]


# Aninhado

matriz = [[i*j for j in range(3)] for i in range(3)]


# Dict comprehension

nomes = ["ana", "bia", "carlos"]

tamanhos = {n: len(n) for n in nomes}


# Set comprehension

unicos = {x % 5 for x in range(20)}
```

## 2. Módulos e Imports

## import **MÓDULOS**

```
# Importações
import os
import json
from datetime import datetime, timedelta
from pathlib import Path

# Módulo próprio (utils.py)
# from utils import minha_funcao

# Variável especial
if __name__ == "__main__":
    # Executa apenas quando rodado diretamente
    print("Executando como script principal")
```

 Python — Módulo 04

# Estruturas de Dados: Listas, Dicts, Tuplas e Sets

Domine as estruturas de dados nativas do Python para manipular coleções de forma eficiente.

## 1. Listas

### list MUTÁVEL

```
# Criação e manipulação
frutas = ["maçã", "banana", "uva", "manga"]

# Acesso e slicing
print(frutas[0])      # maçã
print(frutas[-1])     # manga
print(frutas[1:3])    # ['banana', 'uva']
print(frutas[:2])     # ['maçã', 'uva']

# Métodos principais
frutas.append("kiwi")  # adiciona ao final
frutas.insert(1, "laranja") # insere na posição
frutas.remove("banana") # remove por valor
frutas.pop()          # remove último
frutas.sort()          # ordena in-place
frutas.reverse()       # inverte

# Desempacotamento
primeiro, *meio, ultimo = frutas
```

## 2. Dicionários

**dict** CHAVE-VALOR

```
# Criação
servidor = {
    "nome": "web-01",
    "ip": "10.0.1.50",
    "porta": 8080,
    "ativo": True,
    "tags": ["prod", "api"]
}

# Acesso seguro
print(servidor.get("ip"))           # 10.0.1.50
print(servidor.get("regiao", "us-east-1")) # default

# Métodos úteis
servidor["cpu"] = 4                  # adiciona/atualiza
del servidor["porta"]                # remove
print(servidor.keys())               # chaves
print(servidor.values())             # valores
print(servidor.items())              # pares (k, v)

# Iteração
for chave, valor in servidor.items():
    print(f"{chave}: {valor}")

# Merge (Python 3.9+)
config = {"debug": True} | servidor
```

### 3. Tuplas e Sets

## tuple

**IMUTÁVEL**

```
# Tuplas são imutáveis
coordenadas = (23.5505, -46.6333)
rgb = (255, 128, 0)

# Desempacotamento
lat, lon = coordenadas

# Named tuple
from collections import namedtuple
Servidor = namedtuple("Servidor", ["nome", "ip", "porta"])
srv = Servidor("web-01", "10.0.1.50", 8080)
print(srv.nome) # web-01
```

## set

**ÚNICOS**

```
# Sets: elementos únicos, sem ordem
ips_prod = {"10.0.1.1", "10.0.1.2", "10.0.1.3"}
ips_dev = {"10.0.1.2", "10.0.1.4"}

# Operações de conjuntos
print(ips_prod | ips_dev) # união
print(ips_prod & ips_dev) # interseção
print(ips_prod - ips_dev) # diferença

# Remover duplicatas
logs = ["erro", "info", "erro", "warn", "info"]
unicos = list(set(logs))
```

 Python — Módulo 05

# Arquivos e Exceções: CSV, JSON, try/except

Leitura e escrita de arquivos, manipulação de dados estruturados e tratamento robusto de erros.

## 1. Manipulação de Arquivos

open / with **ARQUIVOS**

```
# Leitura (sempre use 'with' para fechar automaticamente)
with open("config.txt", "r", encoding="utf-8") as f:
    conteudo = f.read()          # tudo de uma vez

    # ou linha por linha:
    # for linha in f:
    #     print(linha.strip())

# Escrita
with open("saida.txt", "w") as f:
    f.write("Linha 1\n")
    f.write("Linha 2\n")

# Append
with open("log.txt", "a") as f:
    f.write(f"[{datetime.now()}] Evento registrado\n")

# pathlib (moderno)
from pathlib import Path
arquivo = Path("dados/config.json")
arquivo.parent.mkdir(parents=True, exist_ok=True)
arquivo.write_text('{"key": "value"}', encoding="utf-8")
```

## 2. CSV e JSON

**CSV** **DADOS**

```
import csv

# Leitura CSV
with open("servidores.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["nome"], row["ip"])

# Escrita CSV
dados = [
    {"nome": "web-01", "ip": "10.0.1.1"},
    {"nome": "web-02", "ip": "10.0.1.2"},
]
with open("output.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["nome", "ip"])
    writer.writeheader()
    writer.writerows(dados)
```

**json** **DADOS**

```
import json

# Leitura JSON
with open("config.json") as f:
    config = json.load(f)

# Escrita JSON
dados = {"nome": "app", "versao": "2.0", "debug": False}
with open("config.json", "w") as f:
    json.dump(dados, f, indent=2, ensure_ascii=False)

# String JSON
json_str = json.dumps(dados, indent=2)
obj = json.loads(json_str)
```

### 3. Tratamento de Exceções

#### try / except EXCEÇÕES

```
# Tratamento completo
try:
    with open("dados.json") as f:
        dados = json.load(f)
        resultado = dados["valor"] / dados["divisor"]
except FileNotFoundError:
    print("Arquivo não encontrado!")
    dados = {}
except KeyError as e:
    print(f"Chave ausente: {e}")
except ZeroDivisionError:
    print("Divisão por zero!")
except Exception as e:
    print(f"Erro inesperado: {type(e).__name__}: {e}")
else:
    print(f"Sucesso! Resultado: {resultado}")
finally:
    print("Bloco finally sempre executa")

# Levantar exceções
def validar_idade(idade):
    if idade < 0 or idade > 150:
        raise ValueError(f"Idade inválida: {idade}")
    return idade
```

 Python — Módulo 06

# Orientação a Objetos: Classes, Herança e Polimorfismo

OOP permite modelar sistemas complexos com classes, encapsulamento, herança e polimorfismo.

## 1. Classes e Objetos

class **OOP**

```

class Servidor:
    """Representa um servidor na infraestrutura."""

    total_servidores = 0 # atributo de classe

    def __init__(self, nome, ip, porta=8080):
        self.nome = nome # público
        self.ip = ip
        self._porta = porta # protegido (convenção)
        self.__status = "stopped" # privado (name mangling)
        Servidor.total_servidores += 1

    @property
    def porta(self):
        """Getter com @property."""
        return self._porta

    @porta.setter
    def porta(self, valor):
        """Setter com validação."""
        if not 1 <= valor <= 65535:
            raise ValueError("Porta inválida")
        self._porta = valor

    def iniciar(self):
        self.__status = "running"
        return f"{self.nome} iniciado em {self.ip}:{self._porta}"

    def __str__(self):
        return f"Servidor({self.nome}, {self.ip})"

    def __repr__(self):
        return f"Servidor('{self.nome}', '{self.ip}', {self._porta})"

# Uso
srv = Servidor("web-01", "10.0.1.50")
srv.porta = 443
print(srv.iniciar())

```

## 2. Herança e Polimorfismo

Herança

oop

```

from abc import ABC, abstractmethod

class RecursoAWS(ABC):
    """Classe abstrata base."""

    def __init__(self, nome, regioao="us-east-1"):
        self.nome = nome
        self.regiao = regioao

    @abstractmethod
    def criar(self):
        pass

    @abstractmethod
    def deletar(self):
        pass

class InstanciaEC2(RecursoAWS):
    def __init__(self, nome, tipo="t3.micro", **kwargs):
        super().__init__(nome, **kwargs)
        self.tipo = tipo

    def criar(self):
        return f"EC2 '{self.nome}' ({self.tipo}) criada em {self.regiao}"

    def deletar(self):
        return f"EC2 '{self.nome}' terminada"

class BucketS3(RecursoAWS):
    def criar(self):
        return f"Bucket '{self.nome}' criado em {self.regiao}"

    def deletar(self):
        return f"Bucket '{self.nome}' deletado"

# Polimorfismo
recursos = [
    InstanciaEC2("web-server", tipo="t3.small"),
    BucketS3("meu-bucket", regioao="sa-east-1"),
]

```

```
for recurso in recursos:  
    print(recurso.criar()) # cada um executa seu método
```

 Python — Módulo 07

# Bibliotecas Essenciais: regex, datetime, subprocess, pytest

Bibliotecas da standard library que todo desenvolvedor Python precisa dominar.

## 1. Expressões Regulares (re)

re REGEX

```
import re

# Busca simples
texto = "IP do servidor: 192.168.1.100, porta: 8080"
ip = re.search(r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}", texto)
print(ip.group()) # 192.168.1.100

# findall - todas as ocorrências
log = "ERROR auth failed | INFO ok | ERROR timeout | WARN slow"
erros = re.findall(r"ERROR \w+", log)
print(erros) # ['ERROR auth', 'ERROR timeout']

# sub - substituição
limpo = re.sub(r"\d{3}\.\d{3}\.\d{3}-\d{2}", "***CPF***", "CPF: 123.456.789-00")

# Grupos nomeados
padrao = r"(?P<ano>\d{4})-(?P<mes>\d{2})-(?P<dia>\d{2})"
match = re.search(padrao, "Data: 2025-01-15")
if match:
    print(match.group("ano")) # 2025

# Compilar para reutilização
EMAIL_RE = re.compile(r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}")
emails = EMAIL_RE.findall("Contato: admin@empresa.com e dev@startup.io")
```

## 2. datetime e subprocess

**datetime** DATA/HORA

```
from datetime import datetime, timedelta, timezone

# Agora
agora = datetime.now()
utc = datetime.now(timezone.utc)

# Formatação
print(agora.strftime("%d/%m/%Y %H:%M"))
print(agora.isoformat())

# Parse de string
data = datetime.strptime("2025-06-15", "%Y-%m-%d")

# Aritmética
amanha = agora + timedelta(days=1)
semana_passada = agora - timedelta(weeks=1)
diff = datetime(2025, 12, 31) - agora
print(f"Faltam {diff.days} dias")
```

**subprocess** SISTEMA

```
import subprocess

# Executar comando
result = subprocess.run(
    ["ls", "-la", "/var/log"],
    capture_output=True,
    text=True,
    timeout=30
)
print(result.stdout)
print(result.returncode) # 0 = sucesso

# Com shell (cuidado com injeção!)
result = subprocess.run(
    "df -h | grep /dev/sda",
    shell=True,
    capture_output=True,
    text=True
)

# Verificar erro
if result.returncode != 0:
    print(f"Erro: {result.stderr}")
```

### 3. Testes com pytest

**pytest** TESTES

```
# test_calculadora.py
import pytest

def somar(a, b):
    return a + b

def dividir(a, b):
    if b == 0:
        raise ValueError("Divisão por zero")
    return a / b

# Testes básicos
def test_somar():
    assert somar(2, 3) == 5
    assert somar(-1, 1) == 0

def test_dividir():
    assert dividir(10, 2) == 5.0

# Testar exceções
def test_dividir_por_zero():
    with pytest.raises(ValueError, match="Divisão por zero"):
        dividir(10, 0)

# Parametrize - múltiplos cenários
@pytest.mark.parametrize("a,b,esperado", [
    (1, 1, 2),
    (0, 0, 0),
    (-1, -1, -2),
    (100, 200, 300),
])

def test_somar_parametrizado(a, b, esperado):
    assert somar(a, b) == esperado

# Executar: pytest -v test_calculadora.py
```

 Python — Módulo 08

## **Consumo de APIs REST: requests, autenticação, paginação**

Integre sistemas consumindo APIs RESTful com a biblioteca requests.

### **1. Biblioteca requests**

**requests** HTTP

```
import requests

# GET simples
response = requests.get("https://api.github.com/users/octocat")
print(response.status_code) # 200
print(response.json())      # dict com dados

# GET com parâmetros
params = {"q": "python", "sort": "stars", "per_page": 5}
r = requests.get("https://api.github.com/search/repositories", params=params)
repos = r.json()["items"]
for repo in repos:
    print(f"{repo['name']}: ★ {repo['stargazers_count']}")

# POST com JSON
payload = {"title": "Bug fix", "body": "Corrigido erro no login"}
r = requests.post(
    "https://api.exemplo.com/issues",
    json=payload,
    headers={"Content-Type": "application/json"}
)

# Timeout e tratamento de erros
try:
    r = requests.get("https://api.exemplo.com/data", timeout=10)
    r.raise_for_status() # levanta exceção se status >= 400
except requests.exceptions.Timeout:
    print("Timeout na requisição")
except requests.exceptions.HTTPError as e:
    print(f"Erro HTTP: {e.response.status_code}")
```

## 2. Autenticação e Headers

**Auth** **SEGURANÇA**

```
import os

# Bearer Token
token = os.environ.get("API_TOKEN")
headers = {
    "Authorization": f"Bearer {token}",
    "Accept": "application/json"
}
r = requests.get(
    "https://api.exemplo.com/me",
    headers=headers
)

# Basic Auth
r = requests.get(
    "https://api.exemplo.com/data",
    auth=("usuario", "senha")
)

# Session (reutiliza conexão e headers)
session = requests.Session()
session.headers.update({"Authorization": f"Bearer {token}"})
r1 = session.get("/endpoint1")
r2 = session.get("/endpoint2")
```

## Paginação API

```
# Paginação automática
def buscar_todos(url, headers):
    """Busca todas as páginas de uma API."""
    resultados = []
    page = 1

    while True:
        r = requests.get(
            url,
            headers=headers,
            params={"page": page, "per_page": 100}
        )
        r.raise_for_status()
        dados = r.json()

        if not dados:
            break

        resultados.extend(dados)
        page += 1

    return resultados

# Paginação com next link
def paginar_com_links(url, headers):
    resultados = []
    while url:
        r = requests.get(url, headers=headers)
        resultados.extend(r.json())
        url = r.links.get("next", {}).get("url")
    return resultados
```

 Python — Módulo 09

## **Automação AWS com boto3: EC2, S3, Lambda, IAM, CloudWatch**

Automatize a infraestrutura AWS usando Python e a SDK oficial boto3.

### **1. EC2 — Gerenciamento de Instâncias**

**boto3 EC2** **AWS**

```
import boto3

ec2 = boto3.resource("ec2", region_name="us-east-1")
client = boto3.client("ec2", region_name="us-east-1")

# Listar instâncias
for instance in ec2.instances.all():
    nome = next((t["Value"] for t in (instance.tags or []) if t["Key"] == "Name"), "N/A")
    print(f"{nome} | {instance.id} | {instance.state['Name']} | {instance.instance_type}")

# Criar instância
instances = ec2.create_instances(
    ImageId="ami-0c02fb55956c7d316",
    InstanceType="t3.micro",
    MinCount=1,
    MaxCount=1,
    KeyName="minha-chave",
    TagSpecifications=[{
        "ResourceType": "instance",
        "Tags": [{"Key": "Name", "Value": "python-demo"}]
    }]
)

print(f"Instância criada: {instances[0].id}")

# Start/Stop
client.stop_instances(InstanceIds=["i-0123456789abcdef0"])
client.start_instances(InstanceIds=["i-0123456789abcdef0"])
```

## 2. S3 — Armazenamento de Objetos

**boto3 S3** AWS

```
s3 = boto3.client("s3")

# Listar buckets
response = s3.list_buckets()
for bucket in response["Buckets"]:
    print(f"{bucket['Name']} (criado: {bucket['CreationDate']})")

# Upload de arquivo
s3.upload_file("relatorio.pdf", "meu-bucket", "docs/relatorio.pdf")

# Upload com metadados
s3.upload_file(
    "dados.json", "meu-bucket", "data/dados.json",
    ExtraArgs={"ContentType": "application/json", "ServerSideEncryption": "AES256"}
)

# Download
s3.download_file("meu-bucket", "docs/relatorio.pdf", "/tmp/relatorio.pdf")

# Listar objetos
paginator = s3.get_paginator("list_objects_v2")
for page in paginator.paginate(Bucket="meu-bucket", Prefix="logs/"):
    for obj in page.get("Contents", []):
        print(f"{obj['Key']} ({obj['Size']} bytes)")
```

### 3. Lambda — Funções Serverless

**boto3 Lambda** **AWS**

```
import json

lambda_client = boto3.client("lambda", region_name="us-east-1")

# Invocar função Lambda
payload = {"nome": "Ricardo", "acao": "processar"}
response = lambda_client.invoke(
    FunctionName="minha-funcao",
    InvocationType="RequestResponse", # síncrono
    Payload=json.dumps(payload)
)
resultado = json.loads(response["Payload"].read())
print(f"Status: {response['StatusCode']}, Resultado: {resultado}")

# Invocação assíncrona
lambda_client.invoke(
    FunctionName="processar-fila",
    InvocationType="Event", # assíncrono (fire-and-forget)
    Payload=json.dumps({"batch_id": "abc123"})
)

# Listar funções
paginator = lambda_client.get_paginator("list_functions")
for page in paginator.paginate():
    for func in page["Functions"]:
        print(f"{func['FunctionName']} | Runtime: {func['Runtime']} | Memória: {func['MemorySize']}]")
```

## 4. IAM e CloudWatch

IAM AWS

```
iam = boto3.client("iam")

# Listar usuários
users = iam.list_users()
for user in users["Users"]:
    print(f"{user['UserName']} | Criado: {user['CreateDate']}")

# Verificar MFA
for user in users["Users"]:
    mfa = iam.list_mfa_devices(UserName=user["UserName"])
    status = "✅" if mfa["MFADevices"] else "❌"
    print(f"{user['UserName']}: MFA {status}")

# Listar políticas do usuário
policies = iam.list_attached_user_policies(
    UserName="ricardo"
)
for p in policies["AttachedPolicies"]:
    print(p["PolicyName"])
```

**CloudWatch** **AWS**

```
cw = boto3.client("cloudwatch")
logs = boto3.client("logs")

# Métricas de CPU
from datetime import datetime, timedelta
response = cw.get_metric_statistics(
    Namespace="AWS/EC2",
    MetricName="CPUUtilization",
    Dimensions=[{"Name": "InstanceId", "Value": "i-012345"}],
    StartTime=datetime.utcnow() - timedelta(hours=1),
    EndTime=datetime.utcnow(),
    Period=300,
    Statistics=["Average"]
)
for dp in response["Datapoints"]:
    print(f"{dp['Timestamp']}: {dp['Average']:.1f}%")

# Criar alarme
cw.put_metric_alarm(
    AlarmName="CPU-Alta",
    MetricName="CPUUtilization",
    Namespace="AWS/EC2",
    Statistic="Average",
    Period=300,
    EvaluationPeriods=2,
    Threshold=80.0,
    ComparisonOperator="GreaterThanThreshold"
)
```

 Python — Módulo 10

## IA com Amazon Bedrock: Claude, Titan, Embeddings

Integre modelos de IA generativa da AWS em suas aplicações Python usando Amazon Bedrock.

### 1. Invocando Modelos com Bedrock

Bedrock - Claude 

```

import boto3
import json

bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")

# Invocar Claude (Anthropic)
def invocar_claude(prompt, max_tokens=1024):
    body = json.dumps({
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": max_tokens,
        "messages": [
            {"role": "user", "content": prompt}
        ]
    })

    response = bedrock.invoke_model(
        modelId="anthropic.claude-3-sonnet-20240229-v1:0",
        contentType="application/json",
        accept="application/json",
        body=body
    )

    resultado = json.loads(response["body"].read())
    return resultado["content"][0]["text"]

# Uso
resposta = invocar_claude("Explique o que é uma VPC na AWS em 3 frases.")
print(resposta)

# Com contexto de sistema
def invocar_claude_sistema(prompt, sistema="Você é um especialista AWS."):
    body = json.dumps({
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 2048,
        "system": sistema,
        "messages": [{"role": "user", "content": prompt}]
    })

    response = bedrock.invoke_model(
        modelId="anthropic.claude-3-sonnet-20240229-v1:0",
        contentType="application/json",
        accept="application/json",

```

```

        body=body
    )
    return json.loads(response["body"].read())["content"][0]["text"]

```

## 2. Titan e Embeddings

### Titan Text IA

```

# Invocar Amazon Titan
def invocar_titan(prompt):
    body = json.dumps({
        "inputText": prompt,
        "textGenerationConfig": {
            "maxTokenCount": 512,
            "temperature": 0.7,
            "topP": 0.9
        }
    })

    response = bedrock.invoke_model(
        modelId="amazon.titan-text-express-v1",
        contentType="application/json",
        accept="application/json",
        body=body
    )

    resultado = json.loads(response["body"].read())
    return resultado["results"][0]["outputText"]

resposta = invocar_titan("Liste 5 boas práticas de segurança no S3.")
print(resposta)

```

## Embeddings

IA

```
# Gerar embeddings para busca semântica
def gerar_embedding(texto):
    body = json.dumps({
        "inputText": texto
    })

    response = bedrock.invoke_model(
        modelId="amazon.titan-embed-text-v1",
        contentType="application/json",
        accept="application/json",
        body=body
    )

    resultado = json.loads(response["body"].read())
    return resultado["embedding"] # vetor 1536D

# Similaridade entre textos
import numpy as np

emb1 = gerar_embedding("Como criar um bucket S3?")
emb2 = gerar_embedding("Criação de storage no AWS S3")
emb3 = gerar_embedding("Receita de bolo de chocolate")

def similaridade(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(f"S3 vs S3: {similaridade(emb1, emb2):.3f}") # ~0.9
print(f"S3 vs bolo: {similaridade(emb1, emb3):.3f}") # ~0.3
```

### 3. Chatbot com Bedrock (Projeto)

chatbot\_aws.py PROJETO

```

"""
Projeto: Chatbot especialista em AWS usando Bedrock
- Mantém histórico de conversa
- Usa Claude como modelo base
- Contexto de sistema como especialista AWS
- Salva conversas em JSON
"""

import boto3
import json
from datetime import datetime

class ChatbotAWS:
    def __init__(self, modelo="anthropic.claude-3-sonnet-20240229-v1:0"):
        self.bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")
        self.modelo = modelo
        self.historico = []
        self.sistema = (
            "Você é um especialista AWS certificado. "
            "Responda de forma clara e prática, com exemplos de código quando relevante "
            "Sempre mencione boas práticas de segurança e custo."
        )

    def enviar(self, mensagem):
        self.historico.append({"role": "user", "content": mensagem})

        body = json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": 2048,
            "system": self.sistema,
            "messages": self.historico
        })

        response = self.bedrock.invoke_model(
            modelId=self.modelo,
            contentType="application/json",
            accept="application/json",
            body=body
        )

        resultado = json.loads(response["body"].read())
        resposta = resultado["content"][0]["text"]

```

```

        self.historico.append({"role": "assistant", "content": resposta})
        return resposta

    def salvar_conversa(self):
        arquivo = f"conversa_{datetime.now():%Y%m%d_%H%M%S}.json"
        with open(arquivo, "w", encoding="utf-8") as f:
            json.dump(self.historico, f, indent=2, ensure_ascii=False)
        print(f"📄 Conversa salva em {arquivo}")

if __name__ == "__main__":
    bot = ChatbotAWS()
    print("🤖 Chatbot AWS (digite 'sair' para encerrar)\n")
    while True:
        pergunta = input("Você: ")
        if pergunta.lower() in ("sair", "exit", "quit"):
            bot.salvar_conversa()
            break
        resposta = bot.enviar(pergunta)
        print(f"\n🤖 {resposta}\n")

```